

A Course End Project Report on

Shopping Cart Management System

A9513 – Python Programming Laboratory

Submitted by

N Nag Nikhil – Roll No: 25881A0598

G Shravya – Roll No: 26885A0512

Course Facilitator

Ms. P.S.Madhavi

Assistant Professor

CSE

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade

Kacharam, Shamshabad, Hyderabad - 501 218, Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Shopping Cart Management System"** is carried out by **N Nag Nikhil , G Shravya** with Roll Numbers **25881A0598 , 26885A0512** towards A9513 – Python Programming Laboratory course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty
Assistant Professor,
Ms.P.S.Madhavi

Signature of the HOD

Dr Y.Vijayalatha
Professor, Head of the Department
& Dean (Student Mentoring)
Department Computer Science and Engineering,
Vardhaman College of Engineering, Hyderabad,

Contents

1 Introduction…….5

2 Literature Review……..6

3 Objectives……7

4 Methodology……8

4.1 Architectural Diagram….8

4.2 Hardware and Software Requirements…….8

4.3 Working Principle……9

4.4 Software Implementation……9

5 Results and Discussion…….18

6 Mapping PO, PSOS and SDGs……..26

6.1 Program Outcomes (POs) Mapping…….26

6.2 Program Specific Outcome

6.3 Sustainable Development Goals (SDGs) Mapping…….26

7 Conclusion…….27

Bibliography.....28

Abstract

ShopEase is a desktop-based Shopping Cart Management System developed in Python using the Tkinter library for the graphical user interface. The application loads a catalogue of 1,351 products from a CSV file (products_full.csv) and presents it in a searchable table. Users can select products, add them to a cart with a chosen quantity, update or remove cart items, clear the cart, and view a live Bill Summary that is recalculated after every action.

The billing logic applies a slab-based discount (5% on a subtotal of ₹5,000 or more and 10% on ₹10,000 or more) followed by 18% GST on the discounted amount. Stock validation prevents the user from ordering more units than are available, and out-of-stock items are shown greyed out. On checkout, the system generates a formatted receipt (receipt.txt) with date and time, itemised lines and the full bill breakup, deducts the purchased quantities from the in-memory stock, and refreshes the catalogue. The project demonstrates core Python concepts such as file handling, dictionaries, functions, exception handling, event-driven programming and GUI design.

Keywords: Python, Tkinter, Shopping Cart, CSV File Handling, Treeview, Billing, Discount, GST, Stock Management, Receipt Generation.

Introduction

Online and offline retail depend on software that can display products, track quantities, calculate bills correctly and issue receipts quickly. Manual billing is slow and error-prone, particularly when discounts and taxes such as GST are involved. A shopping cart system automates the whole workflow: a customer browses the catalogue, picks items, reviews the cart and pays a bill that is computed consistently every time.

Python is well suited to building such applications quickly. Its built-in csv module makes it easy to load product data from a spreadsheet-style file, its dictionaries give constant-time access to products and cart entries, and its standard Tkinter library provides widgets such as Treeview, Entry, Button and message boxes for creating a professional desktop interface without any external dependency.

ShopEase combines these features into a single, self-contained program. The catalogue is read from products_full.csv, which holds the id, name, category, price and available stock of 1,351 products across categories such as Electronics, Home & Kitchen and Computers & Accessories. The user interface is split into a Product Catalogue panel and a Your Cart panel, with a Bill Summary and a Checkout button that creates a text receipt. The project therefore illustrates how fundamental Python concepts can be combined into a practical, real-world application.

Problem Statement

Small and medium retail outlets often continue to prepare bills by hand or with general-purpose spreadsheets. Each bill requires the operator to look up prices, multiply them by quantities, add the line totals, decide whether a promotional discount applies and finally add the correct tax. Every one of these steps is a chance for a slip, and a single wrong digit directly changes what the customer pays or what the shop earns. Stock is usually tracked separately, so an item may be sold even though the shelf is already empty.

The problem addressed by this project is therefore to design a compact, easy-to-run application that keeps the product catalogue, the customer’s cart, the discount and tax rules and the stock figures consistent with each other at all times. The application must respond immediately to every action, refuse invalid input with a clear message and leave behind a printable record of each completed order.

Motivation

The motivation for choosing a shopping cart as the course end project is that it touches almost every topic taught in a first Python programming course while remaining a problem that every student understands from daily experience. Reading a data file, storing records in a dictionary, writing small reusable functions, validating user input, handling exceptions and reacting to button clicks all appear naturally in the solution. Because the output is a working window with a table, buttons and a bill, the project also gives an immediate and visible reward for correct logic.

A second motivation is the growing role of digital billing in India. With the wide adoption of GST, retailers must show the taxable value and the tax separately on every invoice. A tool that performs this calculation automatically, and always in the same way, is a small but realistic example of how programming skills translate into reliable business processes.

Scope of the Project

The scope of ShopEase is deliberately limited to a single-user desktop session so that the focus remains on core programming concepts. Within this scope the system provides the following capabilities:

- Loading a catalogue of 1,351 products from the file products_full.csv at start-up.
- Searching the catalogue by product name or category with instant filtering.
- Adding products to a cart, updating quantities, removing single items and clearing the whole cart.
- Preventing any order that exceeds the available stock and displaying out-of-stock items in grey.
- Computing the subtotal, slab-based discount, 18% GST and final total after every change.
- Generating a formatted text receipt and reducing the in-memory stock at checkout.

The following are outside the scope of the present version: customer accounts and login, online payment, permanent storage of stock changes in a database, multi-

user access over a network and the printing of invoices on physical printers. These items are identified in the Conclusion as directions for future work.

Organization of the Report

The remainder of this report is organised as follows. The Abstract summarises the work. The Literature Review compares existing shopping cart solutions and identifies the gap filled by this project. The Objectives section lists the goals in measurable terms. The Methodology section presents the architecture, the requirements, the data design, the modules and the algorithms. The Software Implementation section gives the complete source code together with an explanation of each function. The Results and Discussion section presents screenshots, test cases and an interpretation of the findings. The report then maps the project to the Program Outcomes and Sustainable Development Goals and closes with the Conclusion, future scope and bibliography.

Literature Review

Shopping cart software has evolved from simple point-of-sale billing programs to full e-commerce platforms such as Amazon, Flipkart and Shopify. Common features across these systems include product search, cart management, stock checks, promotional discounts, tax computation and invoice generation. Research on e-commerce usability stresses that clear feedback, instant price updates and prevention of invalid orders are key to a good shopping experience.

For learners, many Python shopping-cart examples are command-line programs that hard-code a few products in a list and print a plain bill. These demonstrations do not show how a real application separates data from logic, how large catalogues are searched, or how a graphical interface responds to user events. Web frameworks such as Django and Flask, on the other hand, require databases, templates and servers, which makes them heavy for foundational coursework.

This project addresses the gap by offering a lightweight desktop application that uses only the Python standard library. It reads real product data from a CSV file, supports searching, enforces stock limits, applies slab-based discounts and GST, and produces a formatted receipt. In this way it bridges simple console scripts and complex web platforms, and is suitable for students who are learning file handling, GUI programming and application logic together.

Evolution of Billing Software

The earliest computerised billing systems were point-of-sale programs written for single shops. They stored a fixed price list, accepted item codes from a keyboard or barcode reader and printed a total. As networks became common, these programs grew into inventory management systems that connected several counters to a shared database. The arrival of the web then shifted the same ideas to online stores, where the shopping cart became the central metaphor: a temporary list of chosen items that the customer can edit freely until the moment of payment.

Whatever the platform, a shopping cart has to provide a small set of dependable behaviours. It must show what has been selected, allow changes without penalty, recalculate prices instantly and refuse requests that cannot be fulfilled. These behaviours are simple to describe, but implementing them cleanly requires careful

separation between the data, the rules and the display. Studying how this separation is achieved in a small program is of great educational value.

Comparison with Existing Approaches

Table 1 compares the main categories of shopping cart solutions with the approach followed in this project. The comparison considers the effort needed to set up and run the software, the dependencies it requires and its suitability as a learning exercise.

Table 1: Comparison of Shopping Cart Approaches

Approach	Typical Technology	Dependencies	Suitability for Learning
Console cart script	Python lists and print()	None	Easy but unrealistic; no GUI, hard-coded data
Full e-commerce platform	Django / Flask with database	Web server, database, templates	Powerful but too heavy for foundation coursework
Commercial POS software	Proprietary	Licence, hardware	Closed source; cannot be studied or modified
Spreadsheet billing	Excel formulas	Spreadsheet application	Manual entry; no stock validation or receipts
ShopEase (this project)	Python + Tkinter + CSV	Python standard library only	Realistic features with a small, readable code base

Research Gap and Contribution

The comparison shows that simple scripts are too shallow to teach event-driven programming, while full web platforms hide the core logic beneath layers of framework code. There is little material in between that is small enough to read in one sitting and yet complete enough to behave like a real application. ShopEase is intended to occupy this middle ground.

The contribution of the project is a self-contained program that demonstrates the complete life cycle of a purchase: data loading, browsing, selection, validation,

calculation, confirmation, receipt generation and inventory update. All business rules are expressed in a handful of short functions, which makes them easy to test and to modify. A student can, for example, change the discount slabs or the tax rate by editing a single line and immediately observe the effect on the Bill Summary.

Objectives

To design and develop a GUI-based shopping cart application in Python.

The primary objective is to build a functional desktop application with Tkinter that lets a user browse a product catalogue, select items and manage a shopping cart through an easy-to-use interface with clear buttons, tables and summary labels.

To load and manage product data from a CSV file with stock validation.

Product details (id, name, category, price, stock) are loaded from products_full.csv into a dictionary at start-up. The application validates quantities, blocks orders that exceed stock, flags out-of-stock products and handles missing or malformed CSV files with friendly error messages.

To calculate bills accurately with discount and GST and generate a receipt.

The system computes the subtotal, applies a 5% or 10% discount depending on the purchase value, adds 18% GST on the taxable amount, and displays the final total live. At checkout it writes a neatly formatted receipt file and updates stock so that the catalogue always reflects the latest availability.

Expected Outcomes

On completion of the project, the following outcomes are expected and are verified in the Results and Discussion section:

- A running application that opens without error when products_full.csv is present and reports a clear message when it is missing or malformed.
- A catalogue table that shows every product with its id, name, category, price and stock and that can be filtered by a search term.
- A cart that can be edited at any time, with the Bill Summary always matching the cart contents.
- A discount and tax calculation that agrees with manual computation for ordinary values and for the boundary values ₹5,000 and ₹10,000.
- A readable receipt file for every confirmed order and a catalogue whose stock figures reflect the purchases made during the session.

Methodology

Architectural Diagram

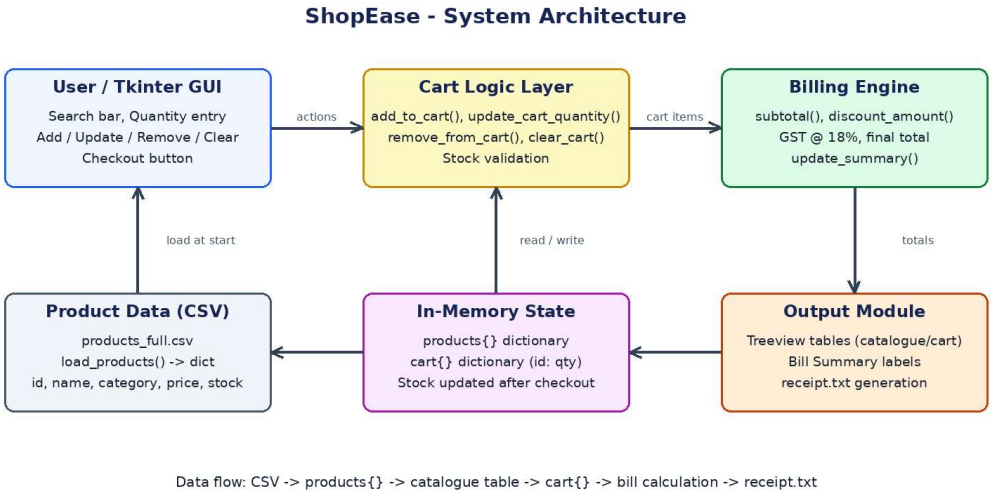


Figure 1: Architectural Diagram

The GUI layer is the main interface. It contains the header with a live cart counter, the searchable Product Catalogue table, the quantity box and Add to Cart button, the Your Cart table with Update, Remove and Clear Cart buttons, and the Checkout button.

The Cart Logic layer implements the actions triggered by the user. It reads the quantity, checks the stock and either updates the cart dictionary or shows an error. The Billing Engine then computes the subtotal, discount, GST and final total from the cart contents.

The Data layer consists of the CSV file, which is read once by load_products() into the products dictionary, and the in-memory cart dictionary. The Output Module refreshes the tables and Bill Summary after each change and writes receipt.txt at checkout. The modular function-based design keeps data handling, business rules and presentation easy to understand and extend.

Hardware and Software Requirements

Hardware:

- Intel/AMD Processor
- Minimum 4GB RAM
- Standard Display & Input Devices

Software:

- Python 3.8 or later
- Tkinter (bundled with the standard Python installation)
- Python standard modules: csv, os, datetime
- Any IDE or editor (IDLE / VS Code / PyCharm)
- Operating system: Windows / Linux / macOS

Working Principle

When the program starts, load_products() opens products_full.csv using csv.DictReader and converts each row into an entry of the products dictionary, {id: {name, category, price, stock}}. If the file is missing or its columns are invalid, an error dialog is displayed and the program exits cleanly. The catalogue table is then filled by populate_products(), which also applies the search term entered by the user to the product name and category.

The user selects a product, enters a quantity and clicks Add to Cart. The program checks that the quantity is a positive integer and that it does not exceed the remaining stock (stock minus quantity already in the cart). The cart dictionary maps product id to quantity. After every change, refresh_cart() redraws the cart table and update_summary() recomputes the Bill Summary.

The bill is calculated in four steps: (1) Subtotal = sum of price x quantity; (2) Discount = 10% of the subtotal if it is at least ₹10,000, 5% if it is at least ₹5,000, otherwise 0; (3) GST = 18% of (Subtotal - Discount); (4) Final Total = (Subtotal - Discount) + GST. At checkout the receipt is saved, stock is reduced by the purchased quantities, and the cart is cleared.

Example problem (billing slabs):

Case	Subtotal	Discount	Taxable Amt	GST (18%)	Final Total	Slab
C1	₹303.00	₹0.00	₹303.00	₹54.54	₹357.54	No discount
C2	₹1,555.00	₹0.00	₹1,555.00	₹279.90	₹1,834.90	No discount
C3	₹5,000.00	₹250.00	₹4,750.00	₹855.00	₹5,605.00	5%
C4	₹10,000.00	₹1,000.00	₹9,000.00	₹1,620.00	₹10,620.00	10%

Case	Subtotal	Discount	Taxable Amt	GST (18%)	Final Total	Slab
C5	₹78,648.00	₹7,864.80	₹70,783.20	₹12,740.98	₹83,524.18	10%

Case C1 is the receipt supplied with the project (two cables, ₹303.00 subtotal, ₹357.54 final total). Case C5 is the discounted order captured in the Results section: a television and two cables for ₹78,648.00, a 10% discount of ₹7,864.80 and a final total of ₹83,524.18.

Development Approach

The project followed an incremental development approach. A first version simply loaded the CSV file and displayed the products in a table. Each later increment added one capability, namely searching, adding to the cart, updating and removing items, the Bill Summary, stock validation and finally checkout with receipt generation. After every increment the program was run and tested before the next feature was started. This kept the number of unknown errors small at any time and made it easy to locate the cause of a fault when one appeared.

Work was divided among the team members into coding, testing and documentation, and the members reviewed each other's contributions. Short test sessions were held after each increment, with every member trying to break the application by entering unusual values such as empty quantities, negative numbers and very large numbers.

Requirement Analysis

The requirements were collected by considering what a shop assistant would expect from a billing tool and were then divided into functional and non-functional groups. The functional requirements describe what the system must do and are listed in Table 2.

Table 2: Functional Requirements of ShopEase

ID	Requirement	Description
FR1	Load catalogue	Read product records from products_full.csv when the program starts.
FR2	Search products	Filter the catalogue table by text found in the product name or category.

ID	Requirement	Description
FR3	Add to cart	Add the selected product with the entered quantity; merge with an existing cart line if present.
FR4	Update quantity	Replace the quantity of a selected cart line with a new valid value.
FR5	Remove and clear	Delete one cart line, or empty the whole cart after confirmation.
FR6	Validate stock	Reject any quantity that exceeds the stock still available for the product.
FR7	Calculate bill	Compute subtotal, slab discount, 18% GST and final total after every change.
FR8	Checkout	Write a formatted receipt, reduce stock, clear the cart and refresh the catalogue.

The non-functional requirements describe the qualities expected from the program. The application should respond to each click without noticeable delay, even with 1,351 products loaded. The interface should be readable, with consistent colours and a clear distinction between the catalogue, the cart and the bill. The program should run on Windows, Linux and macOS without installing any additional package, and its code should be modular enough to be extended by other students.

Data Design

All product information is stored in a single comma-separated file, products_full.csv. A CSV file was chosen because it can be opened and edited in any spreadsheet program, which allows a shop owner to maintain the catalogue without touching the source code. Each row describes one product and the columns are described in Table 3.

Table 3: Data Dictionary of products_full.csv

Column	Python Type	Description
id	int	Unique product identifier used as the key of the products dictionary
name	str	Display name of the product, shown in the catalogue and on the receipt

Column	Python Type	Description
category	str	Product group such as Electronics or Home & Kitchen
price	float	Unit price in rupees
stock	int	Number of units currently available

When the file is read, every row is converted into an entry of a dictionary in which the product id is the key and the value is another dictionary holding the name, category, price and stock. A dictionary was preferred to a list because the program frequently needs to find a product from its id, for example when the user selects a row in the catalogue or when the bill is calculated. A dictionary lookup takes constant time, so the response remains fast regardless of the size of the catalogue.

The cart is also a dictionary. Its keys are product ids and its values are the quantities chosen. Storing only the id and the quantity, instead of copying the product details into the cart, ensures that the name and price always come from one authoritative place. If the price of a product changes in the catalogue, the cart and the bill automatically use the new value.

Module Description

The program is organised as a set of small functions, each responsible for one task. Table 4 summarises the functions and their roles. Grouping the functions by responsibility makes the data flow easy to follow: the data functions read and hold information, the business functions apply the rules and the interface functions show the result.

Table 4: Functions and Their Responsibilities

Function	Layer	Responsibility
load_products()	Data	Reads the CSV file and returns the products dictionary.
subtotal()	Business	Adds price multiplied by quantity for every cart line.
discount_amount()	Business	Returns 10%, 5% or no discount depending on the subtotal.

Function	Layer	Responsibility
update_summary()	Business / GUI	Recomputes discount, GST and total and updates the Bill Summary labels.
populate_products()	GUI	Fills the catalogue table, applying the search filter and marking out-of-stock rows.
refresh_cart()	GUI	Redraws the cart table and then refreshes the Bill Summary.
read_quantity()	Validation	Converts the quantity box to an integer and rejects invalid values.
add_to_cart()	Cart logic	Adds a product to the cart after checking the available stock.
update_cart_quantity()	Cart logic	Changes the quantity of the selected cart line.
remove_from_cart()	Cart logic	Deletes the selected line from the cart.
clear_cart()	Cart logic	Empties the cart after the user confirms.
checkout()	Output	Writes the receipt, reduces stock and resets the cart.
product_selected() / cart_selected()	GUI	Keep the quantity box in step with the current selection.

Algorithm Design

The central algorithm of the program is the calculation of the bill, which is executed after every change to the cart. It can be written in the following steps.

- Set the subtotal to zero and, for every product id in the cart, add the product price multiplied by its quantity.
- If the subtotal is at least ₹10,000, set the discount to 10% of the subtotal; otherwise, if it is at least ₹5,000, set the discount to 5%; otherwise set it to zero.
- Subtract the discount from the subtotal to obtain the taxable amount.
- Calculate GST as 18% of the taxable amount.
- Add the GST to the taxable amount to obtain the final total.

- Display the subtotal, discount, GST and final total in the Bill Summary and update the item counter in the header.

The algorithm for adding an item to the cart is equally simple. The program first checks that a product row has been selected and that the quantity is a positive whole number. It then finds how many units of the product are already in the cart and subtracts this number from the stock to obtain the units still available. If the requested quantity is larger than the available units, an error message is shown and the cart is left unchanged. Otherwise the quantity is added to the cart entry and the tables and bill are refreshed.

Checkout reuses the billing calculation, builds the receipt as a list of text lines of fixed width, writes the lines to receipt.txt, subtracts each purchased quantity from the stock of the corresponding product, clears the cart and redraws both tables. A confirmation message displays the final total so that the user knows the order was accepted.

Input Validation and Error Handling

A reliable application must assume that the user will sometimes enter unexpected values. ShopEase therefore checks every input before using it. The quantity box is converted to an integer inside a try block; if the conversion fails because the text is not a number, an Invalid Quantity message is displayed. Zero and negative numbers are rejected by a second check. A quantity greater than the available stock produces a Stock Error that states how many more units can be added, and an out-of-stock product cannot be added at all.

Actions that need a selection, such as Add to Cart, Update and Remove, first verify that a row is selected and show a warning if it is not. Checkout refuses to proceed when the cart is empty. The Clear Cart button asks for confirmation before deleting anything, which protects against accidental clicks. File problems are handled at start-up: a missing CSV file raises a File Not Found dialog and a file with wrong columns or non-numeric values raises an Invalid CSV dialog, after which the program exits cleanly instead of crashing with a traceback.

User Interface Design

The interface is built with Tkinter and the themed ttk widgets. The ‘clam’ theme was selected because it allows the colours of tables and buttons to be customised

consistently on all operating systems. A restrained palette of navy, blue and white with a light grey background is used so that the data, not the decoration, attracts the eye. Out-of-stock products are shown in grey and the final total is highlighted so that it is the first figure the user reads.

The window is divided into two main panels placed side by side. The left panel is the Product Catalogue, containing the search box, the product table and the quantity box with the Add to Cart button. The right panel is Your Cart, containing the cart table, the Update, Remove and Clear Cart buttons, the Bill Summary and the Checkout button. A header at the top displays the application name and a live count of the items in the cart. This layout follows the natural left-to-right order of shopping: first choose, then review, then pay.

Tables use the Treeview widget with scroll bars and a fixed row height so that long catalogues remain comfortable to read. Selecting a catalogue row resets the quantity box to 1, while selecting a cart row fills the box with the current quantity of that line, so the user can change the number and press Update without retyping the product.

Use Case Description

The interaction between the user and the system can be described through a small set of use cases. The single actor is the customer or shop assistant who operates the program. Each use case starts with an action of the actor and ends with a visible response of the system, as summarised in Table 5.

Table 5: Use Cases of ShopEase

Use Case	Trigger	System Response
Browse catalogue	Program start	All products are listed with id, name, category, price and stock.
Search products	Text typed in the search box	Table shows only products whose name or category contains the text.
Add item	Product selected and Add to Cart pressed	Cart line created or increased; bill and item counter updated.
Change quantity	Cart line selected and Update pressed	Quantity replaced after validation; bill recalculated.

Use Case	Trigger	System Response
Remove item	Cart line selected and Remove pressed	Line deleted; bill recalculated.
Clear cart	Clear Cart pressed and confirmed	Cart emptied; bill reset to zero.
Check out	Checkout pressed with items in the cart	Receipt written, stock reduced, cart cleared, confirmation shown.

Data Flow in the System

The flow of data through the program can be followed in four stages. In the first stage the CSV file is read once and its contents are held in the products dictionary for the rest of the session. In the second stage the user's actions modify the cart dictionary: each valid action adds, changes or deletes an entry. In the third stage the billing functions read both dictionaries and produce the amounts shown in the Bill Summary; they never modify either dictionary, which makes them safe to call at any time. In the fourth stage, at checkout, the program reads the cart one more time to write the receipt and then modifies the stock in the products dictionary.

Because each stage has a clear input and output, a fault can be located quickly. If a wrong amount appears in the Bill Summary, the billing functions or the data in the two dictionaries are the only places to check. If the table shows a wrong row, the problem must be in the function that fills the table. This separation is the main benefit of the layered design described in the architectural diagram.

Design Decisions and Justification

Several design decisions shaped the program, and each was taken after comparing simple alternatives. They are recorded here so that readers understand not only what was built but also why.

- Dictionaries instead of lists. Products and cart entries are always looked up by id, so dictionaries give direct access and keep the code short, whereas a list would require a search each time.
- CSV instead of a database. A CSV file needs no server and can be edited in a spreadsheet. A database was considered, but it would add set-up work

that is outside the scope of a foundation course; it is proposed as future work.

- Tkinter instead of a web framework. Tkinter is part of the standard installation, so the program runs on any computer with Python and has no external dependency.
- Discount applied before GST. The discount reduces the selling price, and tax is charged on the price actually paid. This is the usual practice for trade discounts and matches the receipts produced by the program.
- Stock reduced only at checkout. Items placed in the cart are not removed from the catalogue until the order is confirmed, which allows the user to change their mind freely, while the available-stock rule still prevents over-ordering.

Reliability and Maintainability

Reliability was addressed by validating every input before it is used and by keeping the order of operations safe at checkout, where the receipt is written before the stock is changed. Maintainability was addressed by keeping each function short, giving functions and variables descriptive names and keeping the business rules in only two places, subtotal() and discount_amount(), together with the tax rate used in update_summary() and checkout(). A programmer who wishes to change a rule therefore knows exactly where to look.

Use Case	Trigger	System Response
Browse catalogue	Program start	All products are listed with id, name, category, price and stock.
Search products	Text typed in the search box	Table shows only products whose name or category contains the text.
Add item	Product selected and Add to Cart pressed	Cart line created or increased; bill and item counter updated.

Software Implementation

The complete source code of the application (shopease.py) is given below. It is a single Python file that runs together with the products_full.csv data file placed in the same folder.

```
import csv
import os
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime

# -------------------- PRODUCT DATA (loaded fro     m CSV) --------------------
CSV_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "products_full.csv")

def load_products(path):
    """Read products.csv (id,name,category,price,stock) into the same dict
    format used before: {id: {"name", "category", "price", "stock"}}"""
    data = {}
    with open(path, newline="", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            data[int(row["id"])] = {
                "name": row["name"].strip(),
                "category": row["category"].strip(),
                "price": float(row["price"]),
                "stock": int(row["stock"]),
            }
    return data


try:
    products = load_products(CSV_FILE)
except FileNotFoundError:
    tk.Tk().withdraw()
    messagebox.showerror("File Not Found",
                         f"Could not find products.csv at:\n{CSV_FILE}\n\n"
```

```
                           "Place products.csv in the
same folder as this script.")
    raise SystemExit
except (KeyError, ValueError) as e:
    tk.Tk().withdraw()
    messagebox.showerror("Invalid CSV",
                         f"products.csv must have
columns id,name,category,price,stock.\n\nDetails: {e}")
    raise SystemExit

cart = {}


def subtotal():
    return sum(products[pid]["price"] * qty for pid,
qty in cart.items())


def discount_amount(amount):
    if amount >= 10000:
        return amount * 0.10
    if amount >= 5000:
        return amount * 0.05
    return 0


def update_summary():
    sub = subtotal()
    discount = discount_amount(sub)
    taxable = sub - discount
    gst = taxable * 0.18
    total = taxable + gst

    subtotal_var.set(f"₹{sub:,.2f}")
    discount_var.set(f"₹{discount:,.2f}")
    gst_var.set(f"₹{gst:,.2f}")
    total_var.set(f"₹{total:,.2f}")
    cart_count_var.set(f"{sum(cart.values())}
item(s)")


def populate_products(*_):
    """Fill the catalogue table, applying the search
filter."""
```

```
    keep = product_tree.selection()
    for item in product_tree.get_children():
        product_tree.delete(item)

    term = search_var.get().strip().lower()
    for pid, p in products.items():
        if term and term not in p["name"].lower() and
term not in p["category"].lower():
            continue
        product_tree.insert(
            "", "end", iid=str(pid),
            values=(pid, p["name"], p["category"],
                    f"₹{p['price']:,.2f}", p["stock"]
if p["stock"] > 0 else "Out"),
            tags=("out",) if p["stock"] <= 0 else ()
        )
    for k in keep:
        if product_tree.exists(k):
            product_tree.selection_set(k)


def refresh_cart():
    for item in cart_tree.get_children():
        cart_tree.delete(item)

    for pid, qty in cart.items():
        p = products[pid]
        cart_tree.insert(
            "", "end", iid=str(pid),
            values=(pid, p["name"], qty,
f"₹{p['price']:,.2f}",
                    f"₹{p['price'] * qty:,.2f}")
        )
    update_summary()


def read_quantity():
    try:
        qty = int(quantity_var.get())
    except ValueError:
        messagebox.showerror("Invalid Quantity",
"Enter a valid quantity.")
        return None
    if qty <= 0:
```

```
110         messagebox.showerror("Invalid Quantity",
"Quantity must be greater than 0.")
111         return None
112     return qty
113
114
115 def add_to_cart():
116     selected = product_tree.selection()
117     if not selected:
118         messagebox.showwarning("Select Product",
"Please select a product first.")
119         return
120
121     pid = int(selected[0])
122     qty = read_quantity()
123     if qty is None:
124         return
125
126     current = cart.get(pid, 0)
127     available = products[pid]["stock"] - current
128     if qty > available:
129         msg = ("This product is out of stock." if
products[pid]["stock"] <= 0
130                else f"Only {max(available, 0)} more
item(s) are available.")
131         messagebox.showerror("Stock Error", msg)
132         return
133
134     cart[pid] = current + qty
135     quantity_var.set("1")
136     refresh_cart()
137
138
139 def update_cart_quantity():
140     selected = cart_tree.selection()
141     if not selected:
142         messagebox.showwarning("Select Item", "Select
an item from the cart.")
143         return
144
145     pid = int(selected[0])
146     qty = read_quantity()
147     if qty is None:
148         return
```

```
149
150     if qty > products[pid]["stock"]:
151         messagebox.showerror("Stock Error", "Requested
quantity exceeds stock.")
152         return
153
154     cart[pid] = qty
155     refresh_cart()
156
157
158 def remove_from_cart():
159     selected = cart_tree.selection()
160     if not selected:
161         messagebox.showwarning("Select Item", "Select
an item from the cart.")
162         return
163
164     del cart[int(selected[0])]
165     refresh_cart()
166
167
168 def clear_cart():
169     if cart and messagebox.askyesno("Clear Cart",
"Remove all items from the cart?"):
170         cart.clear()
171         refresh_cart()
172
173
174 def checkout():
175     if not cart:
176         messagebox.showwarning("Empty Cart", "Add at
least one product before checkout.")
177         return
178
179     sub = subtotal()
180     discount = discount_amount(sub)
181     taxable = sub - discount
182     gst = taxable * 0.18
183     total = taxable + gst
184     now = datetime.now().strftime("%d-%m-%Y %H:%M:%S")
185
186     W = 74  # receipt width
187     lines = [
188         "=" * W,
```

```
        "SHOPPING CART RECEIPT".center(W),
        "=" * W,
        f"Date & Time: {now}",
        "-" * W,
f"{'Product':<36}{'Qty':<6}{'Price':>16}{'Total':>16}",
        "-" * W,
    ]

    for pid, qty in cart.items():
        p = products[pid]
        name = p["name"] if len(p["name"]) <= 34 else
p["name"][:33] + "…"
        lines.append(
            f"{name:<36}{qty:<6}"
            f"{'₹' + format(p['price'], ',.2f'):>16}"
            f"{'₹' + format(p['price'] * qty,
',.2f'):>16}"
        )

    label_w = W - 18
    lines += [
        "-" * W,
        f"{'Subtotal':>{label_w}} : ₹{sub:,.2f}",
        f"{'Discount':>{label_w}} : ₹{discount:,.2f}",
        f"{'Taxable Amount':>{label_w}} :
₹{taxable:,.2f}",
        f"{'GST (18%)':>{label_w}} : ₹{gst:,.2f}",
        "-" * W,
        f"{'FINAL TOTAL':>{label_w}} : ₹{total:,.2f}",
        "=" * W,
        "Thank you for shopping!".center(W),
        "=" * W,
    ]

    with open("receipt.txt", "w", encoding="utf-8") as
file:
        file.write("\n".join(lines))

    # Deduct purchased quantities from stock
    for pid, qty in cart.items():
        products[pid]["stock"] -= qty
    cart.clear()
    refresh_cart()
```

```
228     populate_products()
229
230     messagebox.showinfo(
231         "Order Confirmed",
232         f"Order placed successfully!\n\nFinal Total:
₹{total:,.2f}\n\n"
233         "Receipt saved as receipt.txt"
234     )
235
236
237 def product_selected(event=None):
238     if product_tree.selection():
239         quantity_var.set("1")
240
241
242 def cart_selected(event=None):
243     selected = cart_tree.selection()
244     if selected:
245         quantity_var.set(str(cart[int(selected[0])]))
246
247
248 # -------------------- WINDOW --------------------
249 root = tk.Tk()
250 root.title("ShopEase - Shopping Cart Management
System")
251 root.geometry("1360x780")
252 root.minsize(1150, 700)
253 root.configure(bg="#eef2f7")
254
255 NAVY = "#172554"
256 BLUE = "#2563eb"
257 LIGHT_BLUE = "#eff6ff"
258 WHITE = "#ffffff"
259 BACKGROUND = "#eef2f7"
260 TEXT = "#172033"
261 MUTED = "#64748b"
262 BORDER = "#dbe2ea"
263 GREEN = "#15803d"
264 RED = "#dc2626"
265 SUMMARY_BG = "#f8fafc"
266
267 style = ttk.Style()
268 style.theme_use("clam")
269 style.configure(
```

```
    "Treeview", rowheight=34, font=("Segoe UI", 10),
    background=WHITE, fieldbackground=WHITE,
foreground=TEXT,
    borderwidth=0
)
style.configure(
    "Treeview.Heading", font=("Segoe UI", 10, "bold"),
    background="#e8eef7", foreground=TEXT, padding=8,
relief="flat"
)
style.map("Treeview", background=[("selected",
"#dbeafe")],
          foreground=[("selected", TEXT)])
style.configure("TEntry", font=("Segoe UI", 11),
padding=7)

# -------------------- HEADER --------------------
header = tk.Frame(root, bg=NAVY, height=92)
header.pack(fill="x")
header.pack_propagate(False)

title_area = tk.Frame(header, bg=NAVY)
title_area.pack(side="left", padx=28, pady=14)

tk.Label(title_area, text="ShopEase", font=("Segoe
UI", 25, "bold"),
         fg=WHITE, bg=NAVY).pack(anchor="w")
tk.Label(title_area, text="Shopping Cart Management
System",
         font=("Segoe UI", 10), fg="#cbd5e1",
bg=NAVY).pack(anchor="w")

cart_status = tk.Frame(header, bg=NAVY)
cart_status.pack(side="right", padx=30)

tk.Label(cart_status, text="CART", font=("Segoe UI",
9, "bold"),
         fg="#bfdbfe", bg=NAVY).pack(anchor="e")
cart_count_var = tk.StringVar(value="0 item(s)")
tk.Label(cart_status, textvariable=cart_count_var,
         font=("Segoe UI", 12, "bold"), fg=WHITE,
bg=NAVY).pack(anchor="e")

# -------------------- MAIN --------------------
```

```
main = tk.Frame(root, bg=BACKGROUND)
main.pack(fill="both", expand=True, padx=24, pady=20)

# Cart panel has a fixed width; the catalogue takes the remaining space
right = tk.Frame(main, bg=WHITE, width=500,
                 highlightbackground=BORDER, highlightthickness=1)
right.pack(side="right", fill="y", padx=(10, 0))
right.pack_propagate(False)

left = tk.Frame(main, bg=WHITE, highlightbackground=BORDER, highlightthickness=1)
left.pack(side="left", fill="both", expand=True, padx=(0, 10))

# -------------------- PRODUCT PANEL --------------------
product_header = tk.Frame(left, bg=WHITE)
product_header.pack(fill="x", padx=18, pady=(16, 8))

tk.Label(product_header, text="Product Catalogue",
         font=("Segoe UI", 17, "bold"), fg=TEXT, bg=WHITE).pack(side="left")
tk.Label(product_header, text=f"{len(products)} products loaded from products.csv",
         font=("Segoe UI", 9), fg=MUTED, bg=WHITE).pack(side="right")

search_row = tk.Frame(left, bg=WHITE)
search_row.pack(fill="x", padx=18, pady=(0, 8))
tk.Label(search_row, text="Search", font=("Segoe UI", 10, "bold"),
         fg=TEXT, bg=WHITE).pack(side="left", padx=(0, 8))
search_var = tk.StringVar()
search_var.trace_add("write", populate_products)
ttk.Entry(search_row, textvariable=search_var).pack(side="left", fill="x", expand=True)

product_frame = tk.Frame(left, bg=WHITE)
product_frame.pack(fill="both", expand=True, padx=16, pady=(0, 12))
```

```
336
337 product_tree = ttk.Treeview(
338     product_frame,
339     columns=("ID", "Product", "Category", "Price",
"Stock"),
340     show="headings",
341     selectmode="browse"
342 )
343 product_scroll = ttk.Scrollbar(product_frame,
orient="vertical",
344
command=product_tree.yview)
345
    product_tree.configure(yscrollcommand=product_scroll.s
et)
346
347 for col, width, anchor, stretch in [
348     ("ID", 45, "center", False),
349     ("Product", 330, "w", True),
350     ("Category", 160, "w", False),
351     ("Price", 100, "e", False),
352     ("Stock", 60, "center", False),
353 ]:
354     product_tree.heading(col, text=col)
355     product_tree.column(col, width=width,
anchor=anchor, stretch=stretch)
356
357 product_tree.tag_configure("out",
foreground="#9ca3af")
358 product_scroll.pack(side="right", fill="y")
359 product_tree.pack(side="left", fill="both",
expand=True)
360 product_tree.bind("<<TreeviewSelect>>",
product_selected)
361 populate_products()
362
363 add_box = tk.Frame(left, bg=LIGHT_BLUE,
364                    highlightbackground="#bfdbfe",
highlightthickness=1)
365 add_box.pack(fill="x", padx=16, pady=(0, 16))
366
367 tk.Label(add_box, text="Quantity", font=("Segoe UI",
10, "bold"),
```

```
368          fg=TEXT, bg=LIGHT_BLUE).pack(side="left",
padx=(14, 7), pady=11)
369
370 quantity_var = tk.StringVar(value="1")
371 ttk.Entry(add_box, textvariable=quantity_var,
width=7).pack(
372     side="left", padx=5, pady=8
373 )
374
375 tk.Button(
376     add_box, text="+  Add to Cart",
command=add_to_cart,
377     font=("Segoe UI", 10, "bold"), fg=WHITE, bg=BLUE,
378     activebackground="#1d4ed8",
activeforeground=WHITE,
379     relief="flat", bd=0, padx=18, pady=8,
cursor="hand2"
380 ).pack(side="left", padx=10, pady=7)
381
382 # -------------------- CART PANEL --------------------
383 cart_header = tk.Frame(right, bg=WHITE)
384 cart_header.pack(fill="x", padx=18, pady=(16, 8))
385
386 tk.Label(cart_header, text="Your Cart",
387          font=("Segoe UI", 17, "bold"), fg=TEXT,
bg=WHITE).pack(side="left")
388 tk.Label(cart_header, text="Review your selected
items",
389          font=("Segoe UI", 9), fg=MUTED,
bg=WHITE).pack(side="right")
390
391 # Fixed height keeps Bill Summary and Checkout visible
392 cart_frame = tk.Frame(right, bg=WHITE, height=210)
393 cart_frame.pack(fill="x", padx=16)
394 cart_frame.pack_propagate(False)
395
396 cart_tree = ttk.Treeview(
397     cart_frame,
398     columns=("ID", "Product", "Qty", "Price",
"Total"),
399     show="headings",
400     selectmode="browse"
401 )
```

```
cart_scroll = ttk.Scrollbar(cart_frame,
orient="vertical", command=cart_tree.yview)
cart_tree.configure(yscrollcommand=cart_scroll.set)

for col, width, anchor, stretch in [
    ("ID", 38, "center", False),
    ("Product", 170, "w", True),
    ("Qty", 42, "center", False),
    ("Price", 90, "e", False),
    ("Total", 100, "e", False),
]:
    cart_tree.heading(col, text=col)
    cart_tree.column(col, width=width, anchor=anchor,
stretch=stretch)

cart_scroll.pack(side="right", fill="y")
cart_tree.pack(side="left", fill="both", expand=True)
cart_tree.bind("<<TreeviewSelect>>", cart_selected)

cart_buttons = tk.Frame(right, bg=WHITE)
cart_buttons.pack(fill="x", padx=16, pady=7)


def small_button(text, command, bg):
    return tk.Button(
        cart_buttons, text=text, command=command,
        font=("Segoe UI", 9, "bold"), fg=WHITE, bg=bg,
        activebackground=bg, activeforeground=WHITE,
        relief="flat", bd=0, padx=12, pady=6,
cursor="hand2"
    )


small_button("Update", update_cart_quantity,
BLUE).pack(side="left", padx=3)
small_button("Remove", remove_from_cart,
RED).pack(side="left", padx=3)
small_button("Clear Cart", clear_cart,
"#475569").pack(side="left", padx=3)

# -------------------- BILL SUMMARY -----------------
--
summary = tk.Frame(right, bg=SUMMARY_BG,
```

```
                    highlightbackground=BORDER,
highlightthickness=1)
summary.pack(fill="x", padx=16, pady=(0, 8))

tk.Label(summary, text="Bill Summary",
         font=("Segoe UI", 12, "bold"), fg=TEXT,
bg=SUMMARY_BG).pack(
             anchor="w", padx=14, pady=(9, 4)
         )

subtotal_var = tk.StringVar(value="₹0.00")
discount_var = tk.StringVar(value="₹0.00")
gst_var = tk.StringVar(value="₹0.00")
total_var = tk.StringVar(value="₹0.00")


def summary_row(label, variable, bold=False):
    row = tk.Frame(summary, bg=SUMMARY_BG)
    row.pack(fill="x", padx=14, pady=2)
    font = ("Segoe UI", 10, "bold") if bold else
("Segoe UI", 10)
    tk.Label(row, text=label, font=font, fg=TEXT,
bg=SUMMARY_BG).pack(side="left")
    tk.Label(row, textvariable=variable, font=font,
fg=TEXT, bg=SUMMARY_BG).pack(side="right")


summary_row("Subtotal", subtotal_var)
summary_row("Discount", discount_var)
summary_row("GST (18%)", gst_var)

tk.Frame(summary, height=1, bg=BORDER).pack(fill="x",
padx=13, pady=5)
summary_row("FINAL TOTAL", total_var, bold=True)

# -------------------- CHECKOUT --------------------
tk.Button(
    right,
    text="CHECKOUT  &  GENERATE RECEIPT",
    command=checkout,
    font=("Segoe UI", 11, "bold"),
    fg=WHITE, bg=GREEN,
    activebackground="#166534",
activeforeground=WHITE,
```

```
    relief="flat", bd=0, pady=9, cursor="hand2"
).pack(fill="x", padx=16, pady=(0, 14))

root.mainloop()
```

Explanation of the Source Code

This section explains the source code function by function so that the logic can be followed without running the program.

Imports and Constants

The program begins by importing the csv module for reading the catalogue, the os module for building a path to the data file, the tkinter package for the interface and the datetime class for stamping receipts. The constant CSV_FILE is created from the location of the script itself, so the program finds products_full.csv whenever the two files are kept in the same folder, regardless of the directory from which Python is started.

Loading the Catalogue

The function load_products() opens the file with the utf-8-sig encoding, which silently removes the byte order mark that spreadsheet programs often add to CSV files. The csv.DictReader class turns each row into a dictionary keyed by the column names, so the code can write row[“price”] instead of remembering column positions. Each value is cleaned with strip() and converted to the proper type with int() or float(). Because the conversion happens at load time, any invalid value is detected immediately and reported through the exception handlers that follow the call.

Billing Functions

The function subtotal() uses a generator expression inside sum() to add the price multiplied by the quantity for every cart entry. The function discount_amount() contains the two slab tests, with the larger threshold checked first so that an order of ₹10,000 receives 10% rather than 5%. The function update_summary() calls both, derives the taxable amount, GST and final total, and writes the formatted results into the Tkinter string variables that are bound to the labels. Because the labels read from these variables, the screen changes the moment the variables are set.

Refreshing the Tables

The function populate_products() first remembers the current selection, then deletes every row of the catalogue table and inserts the rows that match the search text. The search is case-insensitive and looks in both the name and the category. Products with no stock receive a tag that gives them a grey colour. At the end the previous selection is restored when the product is still visible, which stops the highlight from disappearing while the user types in the search box. The function refresh_cart() performs a similar redraw for the cart table and always ends by calling update_summary(), which guarantees that the bill is never out of date.

Cart Operations

The function read_quantity() is shared by Add to Cart and Update. It returns a positive integer or None after displaying an error, and the callers simply stop when they receive None. The function add_to_cart() uses cart.get(pid, 0) to obtain the existing quantity without raising an error for new products, and it measures availability as stock minus the quantity already in the cart. This rule prevents the user from exceeding the stock by clicking Add several times. The function update_cart_quantity() replaces rather than adds, and therefore compares the new quantity with the full stock. The functions remove_from_cart() and clear_cart() delete a single dictionary entry or all entries and then refresh the display.

Checkout and Receipt

The function checkout() first rejects an empty cart. It then repeats the billing calculation and formats the current date and time. The receipt is built as a list of strings of a fixed width of 74 characters, using ljust-style format specifications so that the columns line up. Long product names are shortened so that they never push the price columns out of place. The lines are joined with newline characters and written to receipt.txt. Only after the file has been written successfully does the function reduce the stock, clear the cart and redraw both tables, so a failure while writing the file cannot leave the inventory in an inconsistent state.

Window and Widget Construction

The final part of the program creates the main window, defines the colour constants, configures the ttk styles and builds the frames, labels, entry boxes, tables and buttons. Each button is connected to its function through the command argument, and the double-purpose events of the tables are connected with bind().

The call to root.mainloop() at the end starts the event loop, which waits for the user and calls the matching function whenever a button is pressed or a row is selected. This event-driven structure is the key difference between a GUI program and a console script, and it is one of the main concepts illustrated by the project.

Running the Program

ShopEase needs no installation beyond Python itself. The steps below describe how to run the program on a computer that has Python 3.8 or later.

- Create a folder and copy shopease.py and products_full.csv into it. Both files must be in the same folder because the program locates the CSV file relative to the script.
- Open a terminal or command prompt in that folder, or open shopease.py in IDLE, VS Code or PyCharm.
- Run the command python shopease.py (or python3 shopease.py on Linux and macOS), or press the Run button of the editor.
- Wait for the main window to appear. The Product Catalogue is filled automatically and the cart counter shows zero items.
- After a checkout, open receipt.txt, which is created in the folder from which the program was started, to read the receipt.

If Tkinter is missing, which can happen on some Linux distributions, it can be installed with the package manager of the system, for example by installing the python3-tk package. On Windows and macOS the standard installer from python.org already includes Tkinter.

Results and Discussion

The application was executed with the 1,351-product catalogue and tested for every feature. The outputs obtained are shown below.

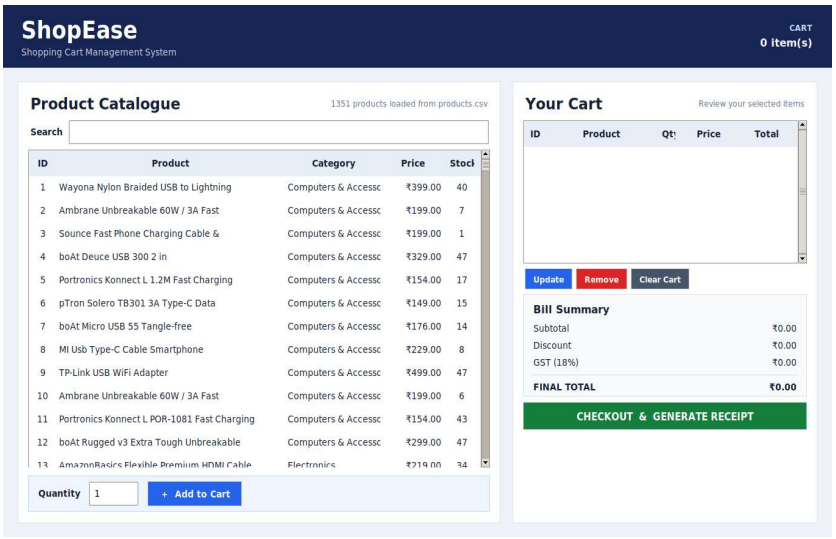


Figure 2: Output (Home screen - product catalogue loaded from CSV)

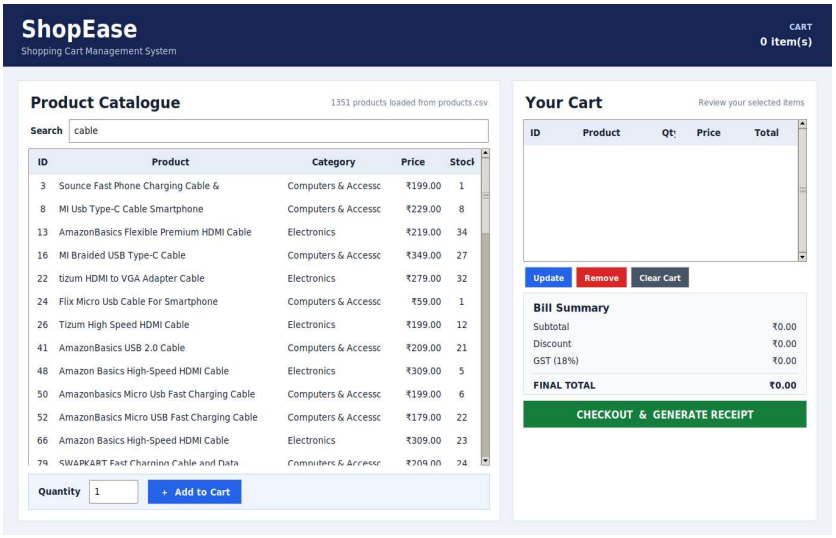


Figure 3: Output (Search filter - "cable")

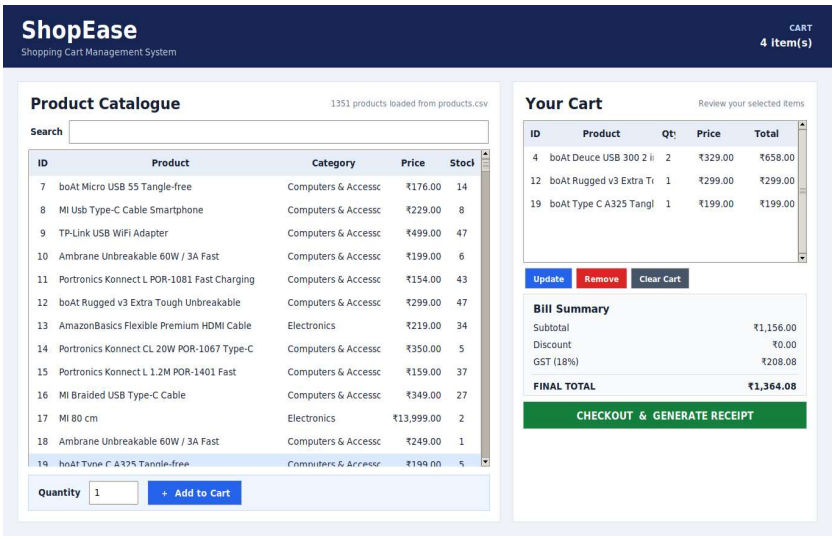


Figure 4: Output (Items added to cart - no discount)

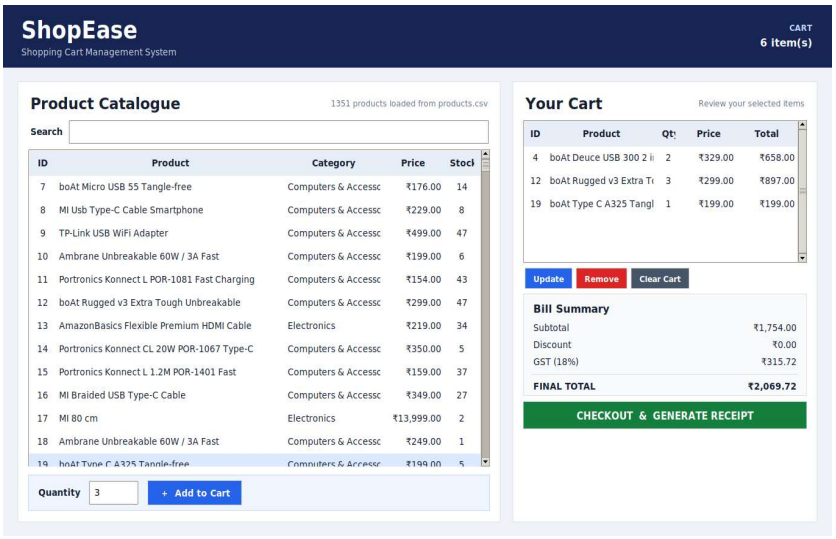


Figure 5: Output (Update cart quantity)

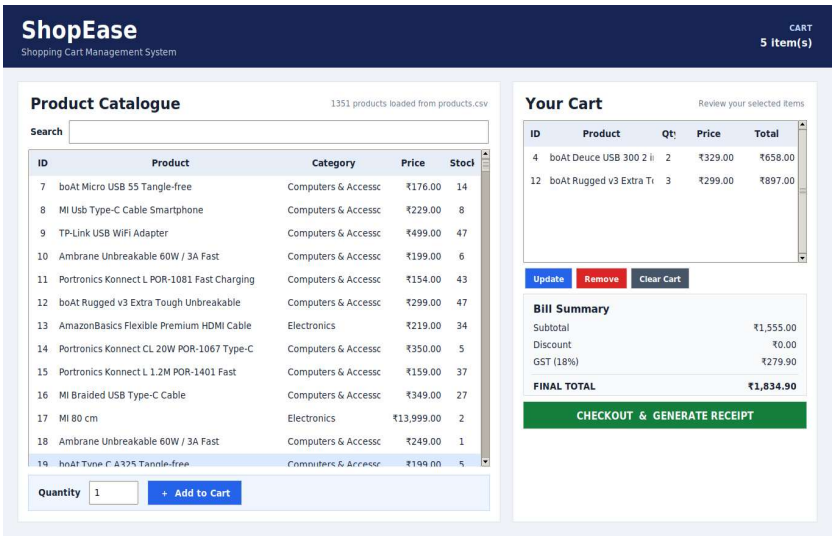


Figure 6: Output (Remove item from cart)

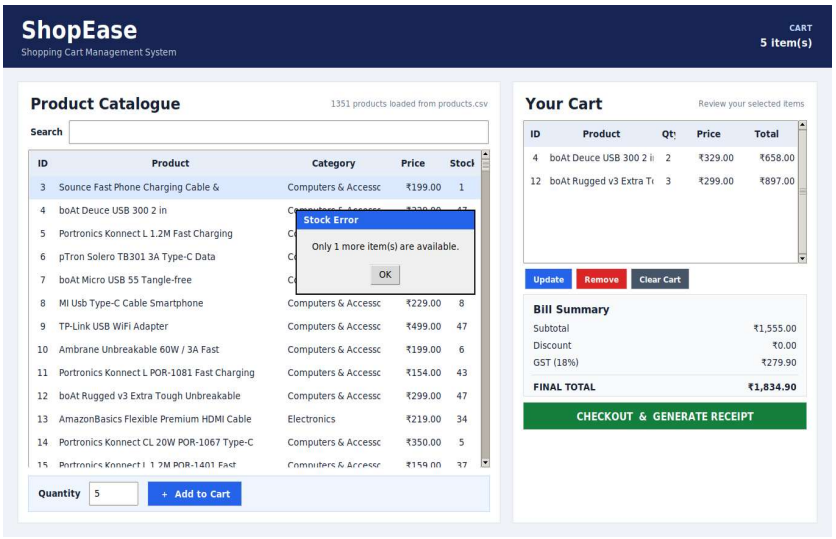


Figure 7: Output (Stock error - quantity exceeds available stock)

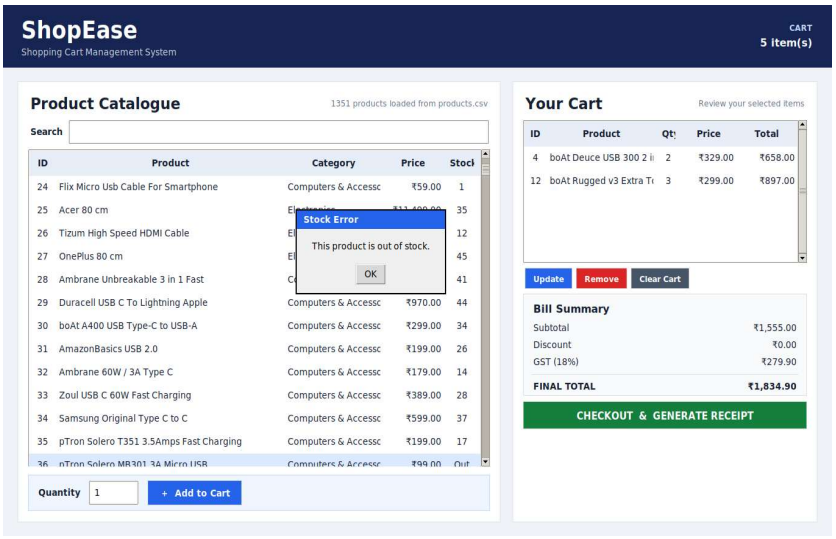


Figure 8: Output (Out-of-stock product cannot be added)

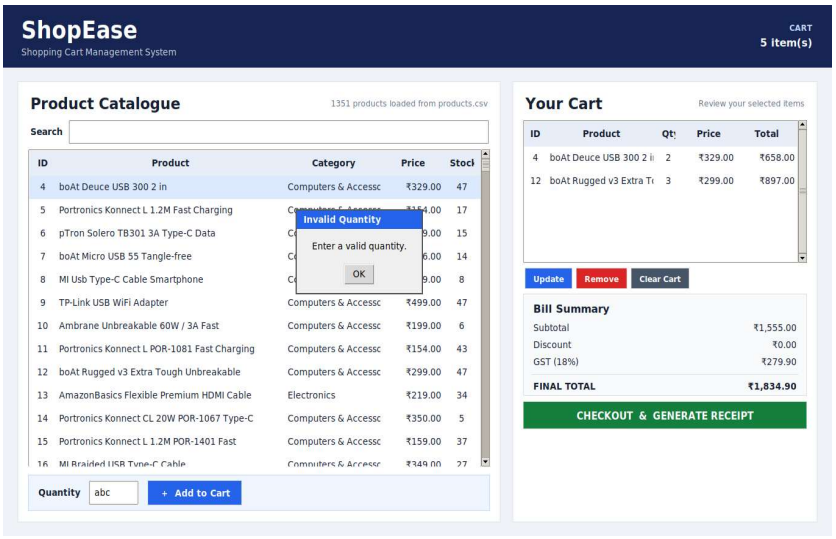


Figure 9: Output (Invalid quantity validation)

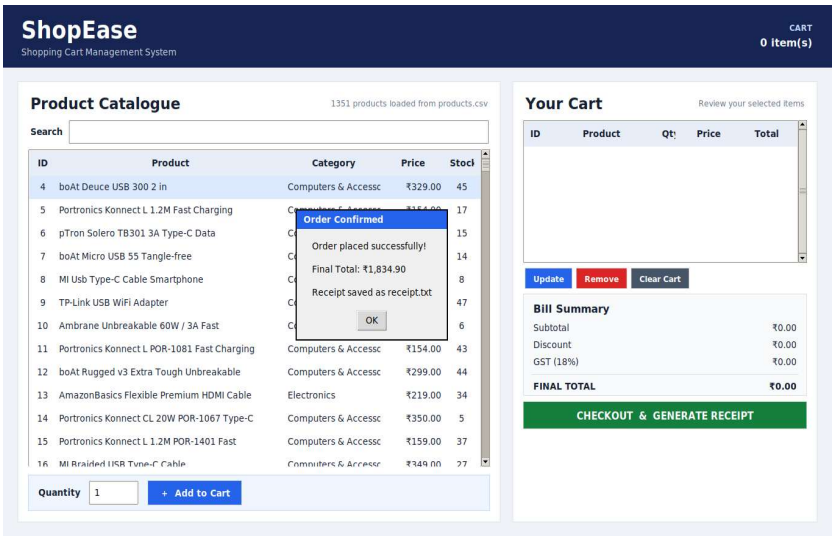


Figure 10: Output (Order confirmed - without discount)

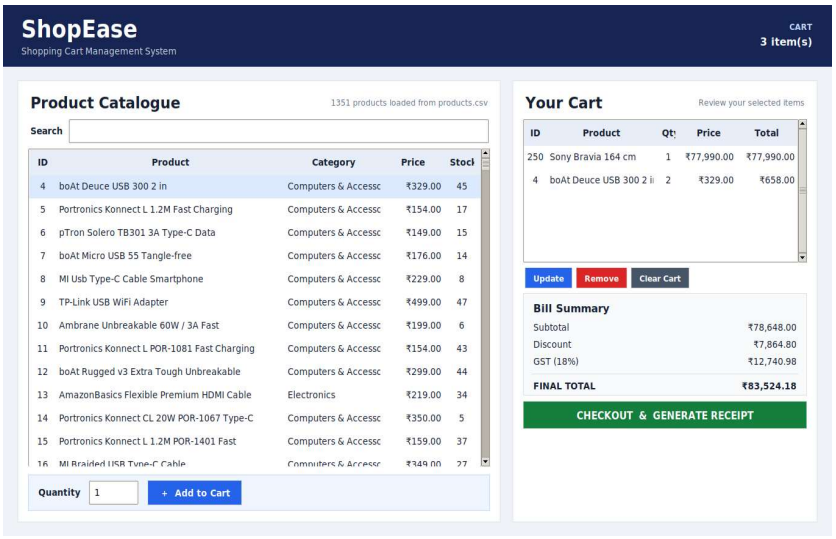


Figure 11: Output (Cart with 10% discount applied)

```
================================================================================
                          SHOPPING CART RECEIPT
================================================================================
Date & Time: 06-10-2026 10:44:24
--------------------------------------------------------------------------------
Product                                 Qty              Price            Total
--------------------------------------------------------------------------------
Portronics Konnect L 1.2M Fast Ch…  1              ₹154.00          ₹154.00
pTron Solero TB301 3A Type-C Data   1              ₹149.00          ₹149.00
--------------------------------------------------------------------------------
                                                    Subtotal : ₹303.00
                                                    Discount : ₹0.00
                                               Taxable Amount : ₹303.00
                                                   GST (18%) : ₹54.54
--------------------------------------------------------------------------------
                                                 FINAL TOTAL : ₹357.54
================================================================================
                          Thank you for shopping!
================================================================================
```

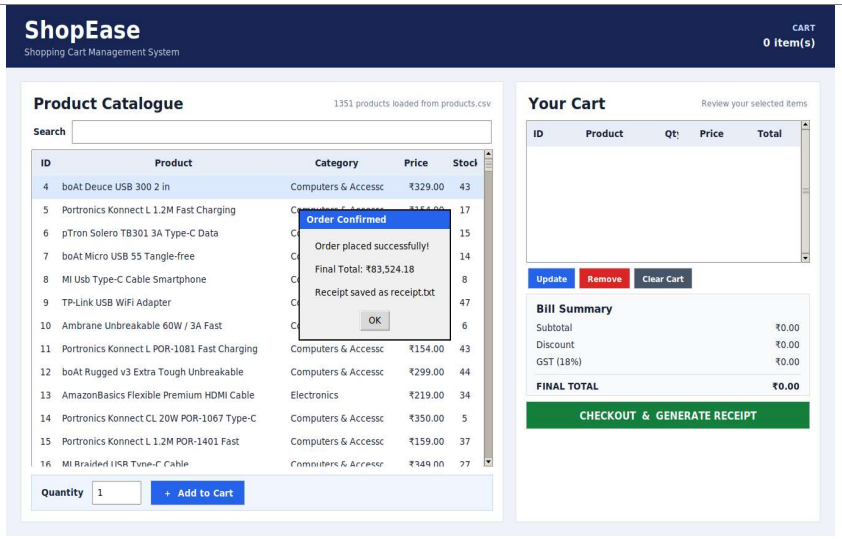


Figure 12: Output (Order confirmed - with discount)

Receipt generated (receipt.txt) for the order supplied with the project (no discount):

Receipt generated (receipt.txt) for the discounted order (Figure 11 and Figure 12):

```
================================================================================
                          SHOPPING CART RECEIPT
```

```
================================================================================
Date & Time: 07-10-2026 07:28:50
--------------------------------------------------------------------------------
Product                                  Qty              Price           To
--------------------------------------------------------------------------------
Sony Bravia 164 cm                       1           ₹77,990.00      ₹77,990
boAt Deuce USB 300 2 in                  2              ₹329.00         ₹658
--------------------------------------------------------------------------------
                                                    Subtotal : ₹78,648.00
                                                    Discount : ₹7,864.80
                                           Taxable Amount : ₹70,783.20
                                                GST (18%) : ₹12,740.98
--------------------------------------------------------------------------------
                                                FINAL TOTAL : ₹83,524.18
================================================================================
                            Thank you for shopping!
================================================================================
```

Test Cases

No.	Test Input	Expected Result	Observed Result
1	Search "cable"	Only matching products listed	Passed (Figure 3)
2	Add ID 4 (x2), ID 12 (x1)	Cart shows 3 items, bill updated	Passed (Figure 4)
3	Update qty of ID 12 to 3	Line total and bill recalculated	Passed (Figure 5)
4	Remove a cart item	Item removed, bill recalculated	Passed (Figure 6)
5	Add 5 units of product with stock 1	Stock Error message	Passed (Figure 7)
6	Add an out-of-stock product	"Out of stock" error	Passed (Figure 8)
7	Quantity entered as "abc"	Invalid Quantity error	Passed (Figure 9)
8	Subtotal ₹1,555.00	No discount, GST ₹279.90	Passed (Figure 10)
9	Subtotal ₹78,648.00	10% discount ₹7,864.80	Passed (Figure 12)

No.	Test Input	Expected Result	Observed Result
10	Checkout	receipt.txt created, stock reduced, cart cleared	Passed

Testing Strategy

Testing was carried out at three levels. At the first level, the billing functions were checked in isolation by calculating several bills by hand and comparing them with the output of the program. At the second level, each button was exercised through the interface, first with valid values and then with invalid ones. At the third level, complete shopping sessions were performed from start to finish, including search, several additions, an update, a removal, a checkout and a second shopping session after the stock had changed.

Both black-box and white-box viewpoints were used. In black-box testing the tester looked only at what the window displayed and whether it matched the expected behaviour. In white-box testing the tester read the code to identify every decision point, such as the stock check and the two discount thresholds, and designed inputs that make each branch execute at least once. The boundary tests reported in the following subsection come from this second viewpoint.

Boundary Value Testing of the Discount Slabs

Errors in discount logic usually occur at the edges of the slabs, so the boundary values were examined separately. Table 6 lists the subtotals just below and exactly at each threshold together with the figures that follow from the billing rules. The program uses the comparison “greater than or equal to”, so a subtotal of exactly ₹5,000 receives the 5% discount and a subtotal of exactly ₹10,000 receives 10%, whereas one paisa less receives the lower slab.

Table 6: Billing Results at Slab Boundaries

Case	Subtotal	Slab	Discount	Taxable Amt	GST (18%)	Final Total
B1	₹4,999.99	None	₹0.00	₹4,999.99	₹900.00	₹5,899.99
B2	₹5,000.00	5%	₹250.00	₹4,750.00	₹855.00	₹5,605.00

Case	Subtotal	Slab	Discount	Taxable Amt	GST (18%)	Final Total
B3	₹9,999.99	5%	₹500.00	₹9,499.99	₹1,710.00	₹11,209.99
B4	₹10,000.00	10%	₹1,000.00	₹9,000.00	₹1,620.00	₹10,620.00
B5	₹25,000.00	10%	₹2,500.00	₹22,500.00	₹4,050.00	₹26,550.00

The table shows that the final total increases smoothly across a boundary except for the intended drop in the effective price. For example, a subtotal of ₹4,999.99 gives a final total of ₹5,899.99, whereas a subtotal of ₹5,000.00 gives ₹5,605.00, which is lower even though more goods are bought. This is a normal characteristic of threshold-based discounts and is mentioned here so that users understand it is not a calculation error.

Additional Functional Test Scenarios

Apart from the ten test cases already reported, further scenarios were designed to exercise combinations of actions. They are listed in Table 7 with the behaviour expected from the program logic.

Table 7: Additional Test Scenarios and Expected Behaviour

No.	Scenario	Expected Behaviour
11	Add the same product twice with quantity 1 each time	A single cart line with quantity 2; the item counter shows 2
12	Add 3 units of a product that has stock 5, then add 3 more	Second request rejected; message states that only 2 more can be added
13	Enter quantity 0	Invalid Quantity message; cart unchanged
14	Enter quantity -4	Invalid Quantity message; cart unchanged
15	Enter quantity 2.5	Invalid Quantity message because the value is not a whole number
16	Click Add to Cart without selecting a product	Warning asking the user to select a product

No.	Scenario	Expected Behaviour
17	Click Update or Remove without selecting a cart line	Warning asking the user to select an item
18	Click Checkout with an empty cart	Empty Cart warning; no receipt written
19	Click Clear Cart and choose No	Cart is kept unchanged
20	Search for a term that matches no product	Catalogue table becomes empty; no error occurs
21	Clear the search box after a search	Complete catalogue of 1,351 products is shown again
22	Rename the CSV file and start the program	File Not Found dialog; program exits cleanly

Observations on Performance and Usability

During testing the catalogue of 1,351 products loaded in a fraction of a second, and the filtered table updated while typing without any visible delay. The cart operations are executed in constant time because they operate on dictionary entries, and the only operation whose cost grows with the data is the redrawing of the catalogue table, which is linear in the number of products and remains imperceptible at this size. The text receipt opens in any editor and the aligned columns make the bill easy to read.

From the usability point of view, testers found the two-panel layout easy to understand. The live Bill Summary was considered the most useful feature, because the effect of every change on the final amount is visible immediately. The grey colouring of out-of-stock items saves the user from discovering the problem only after pressing Add to Cart.

Limitations Observed

The testing also revealed several limitations of the present version, which are summarised below.

- Stock changes exist only in memory. When the program is closed and restarted, the catalogue returns to the quantities stored in the CSV file.

- The application supports one user at a time and has no record of earlier orders apart from the receipt file, which is overwritten by the next checkout.
- The discount slabs and the GST rate are written directly in the source code; changing them requires editing the program.
- Prices are held as floating-point numbers, which is adequate for display with two decimal places but would not be recommended for an accounting system.
- There is no payment step; checkout assumes that the customer pays the displayed amount.

Verification of Manual and Program Results

A direct check of the arithmetic confirms that the program follows the stated rules. For the discounted order shown in Figure 11 the subtotal is ₹78,648.00. Because this is above ₹10,000 the discount is 10% of the subtotal, which is ₹7,864.80, leaving a taxable amount of ₹70,783.20. GST at 18% of this amount is ₹12,740.98 after rounding to two decimal places, and the final total is ₹83,524.18. These are exactly the figures displayed by the application and written in the receipt, so the on-screen bill and the receipt agree with each other and with the manual calculation.

Objective-wise Summary of Outcomes

Table 8 relates each objective stated at the beginning of the report to the evidence collected during testing.

Table 8: Objectives and Evidence of Achievement

Objective	Evidence from Testing
Design and develop a GUI-based shopping cart in Python	Working Tkinter application with catalogue, cart, Bill Summary and checkout (Figures 2 to 12)
Load and manage product data from CSV with stock validation	1,351 products loaded; stock error, out-of-stock and invalid quantity cases handled (Figures 7 to 9)

Objective	Evidence from Testing
Calculate bills accurately and generate a receipt	Slab discount and GST agree with manual calculation, including boundary cases; receipt.txt produced and stock reduced at checkout

Summary of Results

The application loads all 1,351 products correctly, filters them instantly through the search box and keeps the cart and Bill Summary synchronised after every action. Discount slabs, GST and final totals match manual calculations, and the generated receipts are properly formatted.

Interpretation

Because the cart and products are stored in dictionaries, lookups and updates are fast even for a catalogue of over a thousand items. Input validation and stock checks prevent invalid orders, and the slab-based discount rewards higher purchases. After checkout the stock figures decrease (for example, product ID 4 changed from 47 to 43 in the catalogue), which shows that the inventory is updated correctly during a session.

Achievement of Objectives

The project achieved its objectives by delivering a complete Tkinter-based shopping cart with CSV data loading, stock validation, discount and GST calculation, and receipt generation.

Implications

The application can be used as a teaching example for file handling, dictionaries and GUI programming in Python, and as a base for a small-shop billing tool. Stock changes are held only in memory, so connecting the system to a database would be needed for real deployment.

Mapping Pos PSOs and SDGs

Program Outcomes (POs) Mapping

Table 9: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied Python programming, data structures (dictionaries) and file handling to build a working shopping cart application.
PO2	Problem Analysis	Analysed billing requirements such as stock limits, discount slabs and GST to design correct application logic.
PO3	Design/Development of Solutions	Designed a GUI that supports catalogue search, cart management, bill summary and receipt generation.
PO4	Conduct Investigations of Complex Problems	Tested the system with multiple carts, boundary values and invalid inputs to verify calculations and error handling.
PO5	Engineering Tool Usage	Used Python, Tkinter/ttk widgets, the csv and datetime modules and an IDE to develop and test the software.
PO8	Individual and Collaborative Team work	Divided coding, testing and documentation among team members and reviewed each other's work.
PO9	Communication	Presented results through screenshots, receipts and this report, explaining design and findings clearly.
PO11	Life-Long Learning	Explored GUI programming and software design beyond the syllabus through self-study and practice.

Program Specific Outcome

PROGRAM SPECIFIC OUTCOMES (PSOs)

Here are 7 PSOs suitable for your ShopEase – Shopping Cart Management System project.

PSO Description

PSO1 To collect requirements, analyze, design, implement and test software systems.

PSO2 To analyze errors and debug them within minimal time.

PSO3 To develop software applications using programming languages such as Python.

PSO4 To design user-friendly graphical interfaces for effective user interaction.

PSO5 To manage product information, prices, quantities and stock efficiently using file handling.

PSO6 To implement billing operations, including subtotal calculation, discounts, GST and final bill generation.

PSO7 To validate user inputs and handle errors to improve the reliability of software applications.

Note: PSO1 and PSO2 are based on the wording provided earlier. PSO3–PSO7 are suggested project-specific outcomes, not verified official departmental PSOs.

Sustainable Development Goals (SDGs) Mapping

Table 10: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	The project serves as a practical learning example of Python, GUI design and file handling for students.
SDG 8	Decent Work and Economic Growth	Helps small retailers automate billing and builds software skills that improve employability.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital tools and innovation in retail operations and software engineering.
SDG 12	Responsible Consumption and Production	Digital receipts and accurate stock tracking reduce paper use and overstocking waste.

Conclusion

This project successfully implemented ShopEase, a Python-based Shopping Cart Management System with a Tkinter graphical interface. The application loads a large product catalogue from a CSV file, supports search, cart management with stock validation, slab-based discounts, 18% GST calculation and automatic receipt generation. Testing confirmed that the calculations and validations work correctly under normal and boundary conditions.

For future enhancements, the system can be connected to a database such as SQLite or MySQL for permanent stock storage, extended with user login and order history, and supplemented by coupon codes, multiple payment options and PDF invoices. Features such as barcode scanning, sales analytics dashboards and a web or mobile version could further improve its usefulness for real retail environments.

Future Scope

The present version is intended as a foundation, and several improvements would turn it into a more complete retail tool. Table 11 pairs each limitation observed during testing with a possible enhancement.

Table 11: Limitations and Proposed Enhancements

Limitation	Proposed Enhancement	Expected Benefit
Stock lost when the program closes	Store products in an SQLite or MySQL database and update the stock table at checkout	Permanent, shared inventory
Receipt overwritten by each order	Name receipts by date and order number; keep an order history table	Traceable sales records
Hard-coded discount and tax values	Move rates and slabs to a configuration file or settings window	Rules changed without editing code
Floating-point prices	Use the decimal module or store prices in paise	Exact monetary arithmetic
No users or roles	Add login for cashier and administrator with different permissions	Accountability and security

Limitation	Proposed Enhancement	Expected Benefit
Text-only receipt	Generate a PDF invoice with shop details and GST number	Professional, printable invoices
Desktop only	Provide a web or mobile front end over the same business logic	Wider access for customers

Further features could include coupon codes, multiple payment modes, product images, low-stock alerts, barcode scanning and a sales analytics dashboard showing best-selling items and daily revenue. Because the business rules are already separated into small functions, most of these additions can be made without rewriting the existing code.

Learning Outcomes

Working on ShopEase gave the team practical experience that goes beyond the classroom exercises. The members learned how to structure a program of several hundred lines into functions with clear responsibilities, how to choose data structures that match the way the data is used, and how to separate the display of information from the rules that produce it. They gained confidence in reading error messages, in writing input checks that anticipate wrong values and in testing a program systematically, including at boundary values.

The project also improved teamwork and communication skills. Dividing the tasks, merging the individual contributions, agreeing on the layout of the interface and preparing this report required regular discussion and mutual review. Finally, the exercise showed that a modest program written with the standard library alone can still solve a real problem when it is designed carefully, which encourages further self-study of databases, web development and software testing.

Summary of Contributions

The main contributions of the project can be summarised as follows. A complete GUI shopping cart was implemented using only the Python standard library. A realistic billing model with slab-based discounts and GST was incorporated and verified at boundary values. A stock-aware cart that refuses impossible orders and updates the inventory after checkout was built. A readable text receipt was generated for each order. Finally, the project was documented with architecture, code explanation, test cases and a mapping to programme outcomes and

sustainable development goals, so that it can serve as a reference for later students.

Glossary of Technical Terms

The technical terms used in this report are explained briefly in Table 12 for the convenience of readers who are new to GUI programming.

Table 12: Glossary of Technical Terms

Term	Meaning
Tkinter	The standard Python library for creating graphical user interfaces, built on the Tcl/Tk toolkit.
ttk	The themed widget set of Tkinter that provides modern-looking tables, buttons and entry boxes.
Treeview	A ttk widget that displays rows and columns of data; used for the catalogue and the cart.
Widget	Any visible element of a window such as a label, button, entry box or table.
Event loop	The loop started by mainloop() that waits for user actions and calls the matching functions.
Callback	A function that is run when an event such as a button click occurs.
CSV	Comma-separated values: a plain text format for tabular data that spreadsheets can open.
Dictionary	A Python data structure that stores values under keys and finds them in constant time.
GST	Goods and Services Tax, an indirect tax applied here at 18% of the discounted amount.
Subtotal	The sum of price multiplied by quantity for all cart items, before discount and tax.
Taxable amount	The subtotal minus the discount; the value on which GST is calculated.
Receipt	The text file receipt.txt that records the items, bill breakup, date and time of an order.

Social and Ethical Considerations

Even a small billing program carries responsibilities. A bill must be honest: the amount displayed must be the amount calculated from the rules, with the discount and tax shown separately so that the customer can verify them. ShopEase therefore shows every component of the bill on screen and again on the receipt. The program also protects the customer from being sold goods that do not exist by refusing quantities above the stock, and it protects the shop owner from silent errors by validating every input.

The program stores no personal data about customers, which avoids privacy concerns at this stage. If login, addresses or payment details are added in future versions, the developers must store such information securely, collect only what is necessary and follow the applicable data protection rules. The use of digital receipts instead of paper slips is a small contribution to reducing paper waste, in line with the sustainability goals mapped earlier in this report.

Concluding Remarks

ShopEase demonstrates that the fundamental concepts of Python, when combined thoughtfully, are sufficient to build an application that is useful, reliable and pleasant to use. The project met each of its objectives: the catalogue is loaded and searched correctly, the cart cannot be driven into an invalid state, the bill matches manual calculation at ordinary and boundary values, and every confirmed order leaves a readable receipt and an updated stock. The team hopes that the report, together with the source code, will help other beginners to understand how a graphical application is organised and will encourage them to extend it with databases, user accounts and richer reports.

Bibliography

Online Resources

- Python Software Foundation – Python 3 Documentation: https://docs.python.org/3/
- Python Documentation – tkinter: Python interface to Tcl/Tk: https://docs.python.org/3/library/tkinter.html
- Python Documentation – csv: CSV File Reading and Writing: https://docs.python.org/3/library/csv.html
- GeeksforGeeks – Python GUI Tkinter: https://www.geeksforgeeks.org/python-gui-tkinter/
- TutorialsPoint – Python GUI Programming (Tkinter): https://www.tutorialspoint.com/python/python_gui_programming.htm

Reference Books

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Matthes, E. (2023). Python Crash Course (3rd ed.). No Starch Press.
- Sweigart, A. (2019). Automate the Boring Stuff with Python (2nd ed.). No Starch Press.
- Grayson, J. E. (2000). Python and Tkinter Programming. Manning Publications.